



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

7. Recursion

การเขียนโปรแกรมแบบเวียนบังเกิด

ดร.สรร รัตนสัญญา

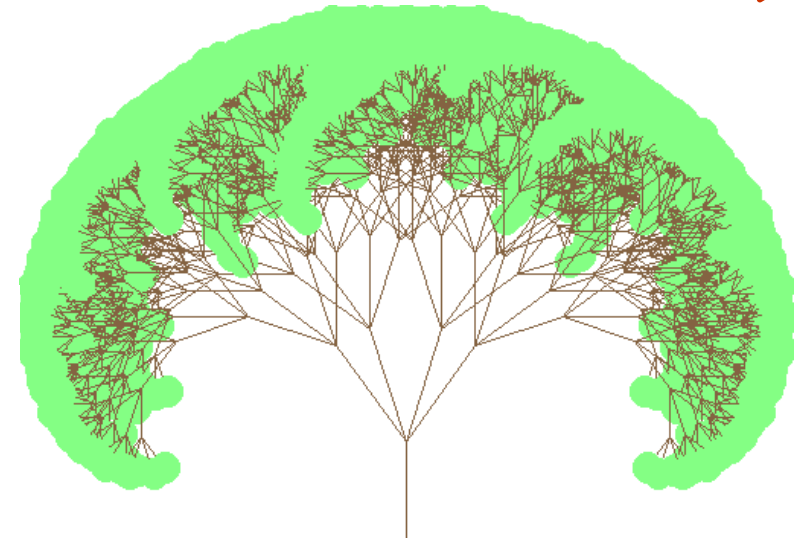
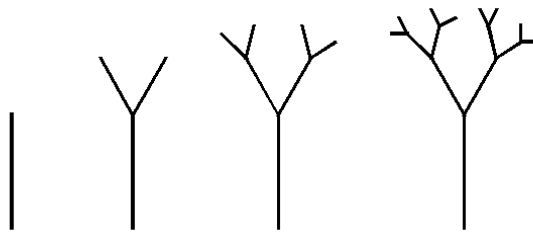
san.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

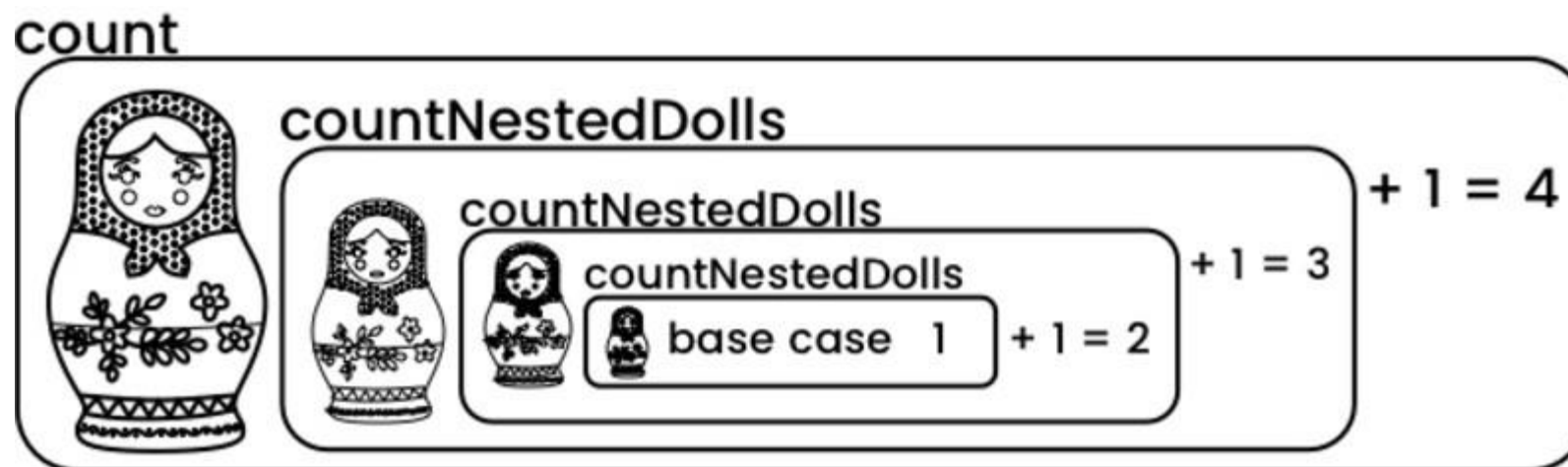
การเวียนบังเกิด (Recursion)

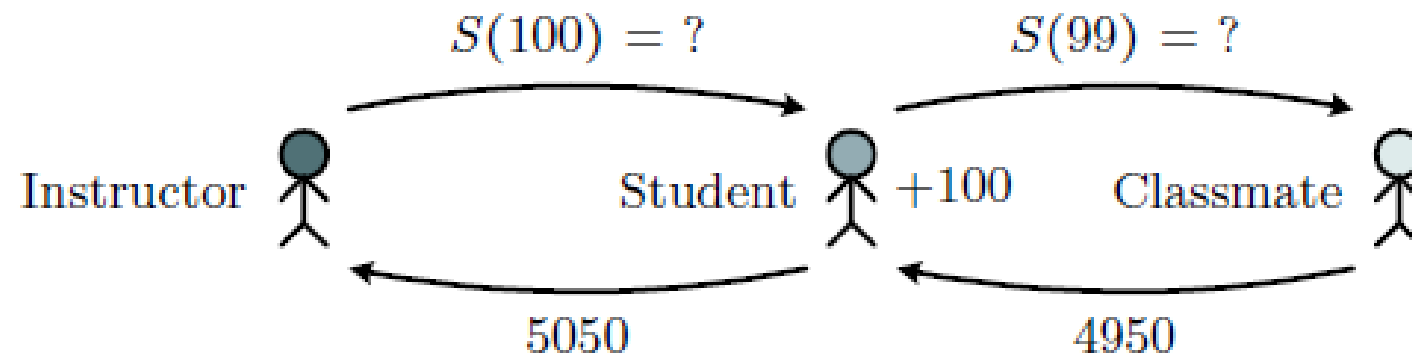
- ปรากฏการณ์ที่มีการกลับไปอ้างอิงถึงตนเอง
- มีนิยามเช่นเดียวกันในลำดับต่ำลงไป
- นิยมใช้ในการแก้ปัญหาต่าง ๆ เพราะเข้าใจง่าย และเป็นธรรมชาติ
- ในทางคอมพิวเตอร์ เรียกว่า ฟังก์ชันเวียนบังเกิด



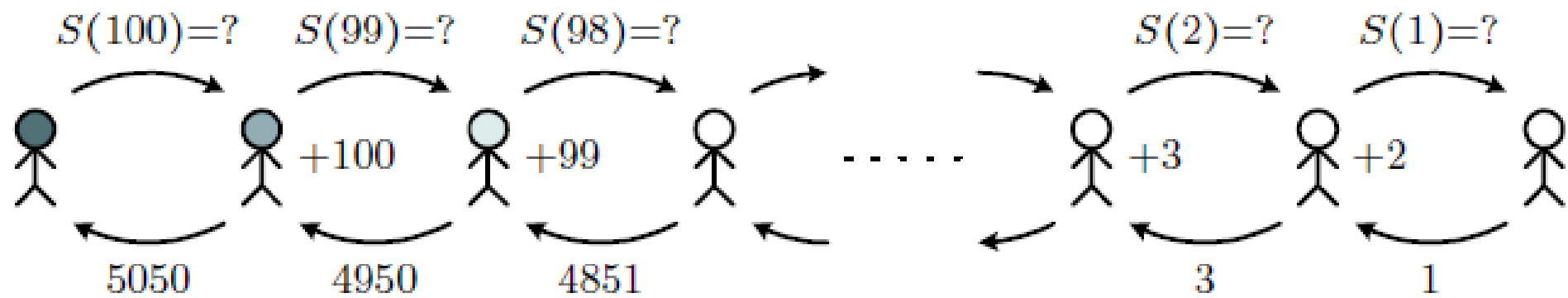
ฟังก์ชันเวียนบังเกิด (Recursive function)

- ฟังก์ชันที่เรียกตัวเองซ้ำไปซ้ำมาจนกว่าจะแก้ปัญหาก็ได้
- แนวคิดในการแก้ปัญหาคือ การแบ่งปัญหานั้นออกเป็นปัญหาย่อย ๆ โดยที่
 - ปัญหาย่อยเหล่านั้นมีโครงสร้างเดียวกันกับปัญหาต้นแบบ ซึ่งทำให้สามารถแก้ปัญหานั้นได้ง่ายกว่า
 - เมื่อนำผลลัพธ์ของปัญหาย่อยเหล่านั้นมารวมกัน จะได้คำตอบของปัญหาต้นแบบ
- อยู่บนพื้นฐานของการอุปนัย หรือการเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)





(a)

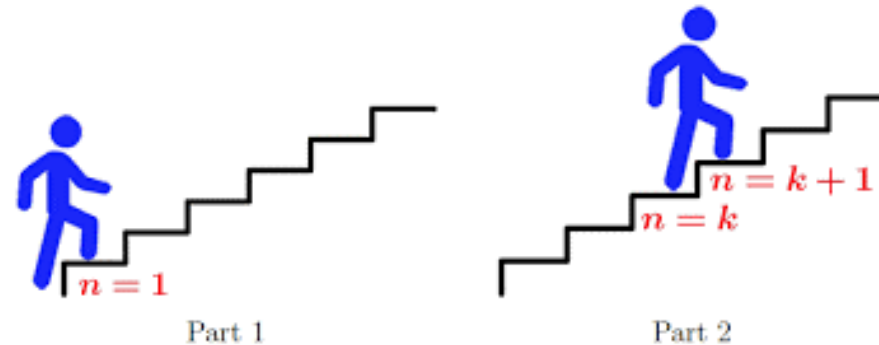


(b)

การเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)

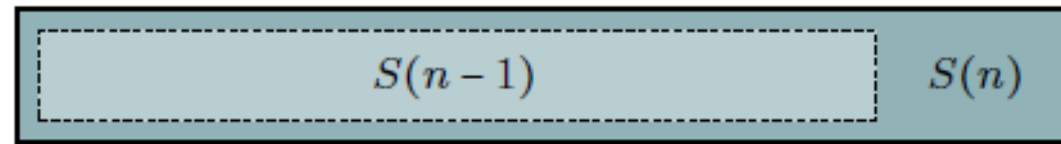
- การอุปนัย หรือ การเหนี่ยวนำทางคณิตศาสตร์ ใช้กับจำนวนเต็มที่ไม่มากกว่า 0

- Base case: ให้ $P(1)$ เป็นจริง

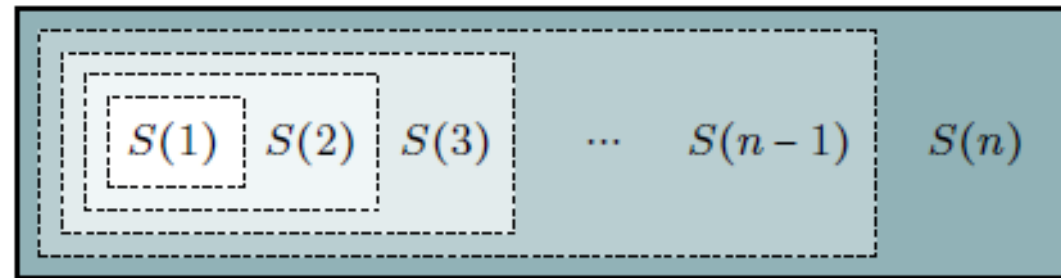


- Inductive case (Recursive case): ให้ $P(2) \dots P(k)$ เป็นจริง พิสูจน์ว่า $P(k+1)$ เป็นจริง



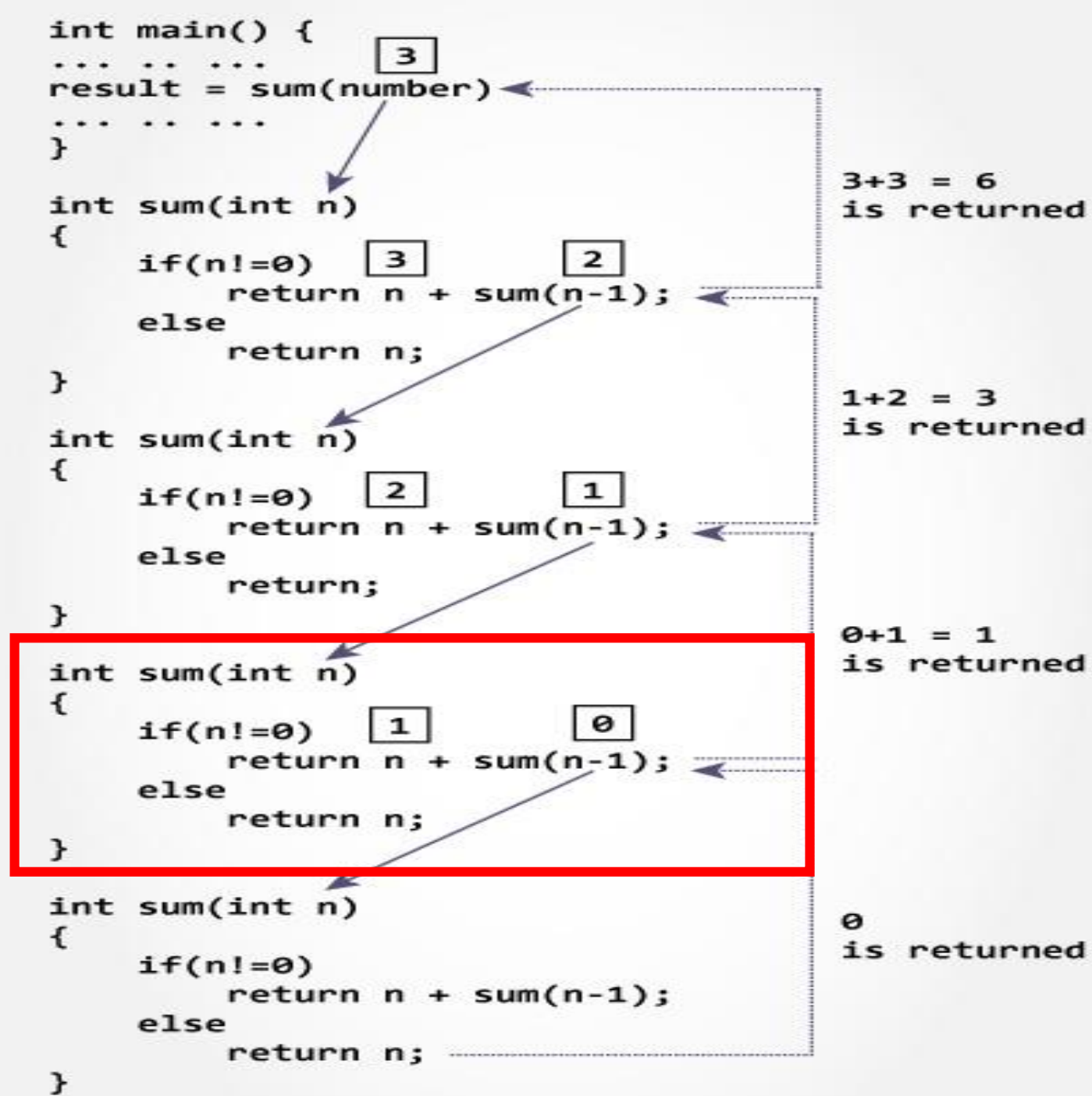


(a)



(b)





สรุปการทำงานของฟังก์ชันเวียนบังเกิด

- ฟังก์ชันหนึ่ง**เรียกใช้งานตัวเอง** โดยแต่ละครั้งที่ตัวมันเองถูกเรียก
- ปัญหาจะถูกแบ่งเป็น**ปัญหาย่อย (Recursive case)** ที่มี**ขนาดเล็กลง**
- และจะเกิดค่าตัวแปรหรือพารามิเตอร์**เปลี่ยนแปลง**ไปอย่างอัตโนมัติ
- แล้วกำหนด**การทำงานขั้นสุดท้าย (Base case)** ไว้ เมื่อทำงานถึงขั้นสุดท้ายก็จะสิ้นสุดการทำงานและส่งผลลัพธ์กลับไป

- เมื่อมีกรณีอย่างน้อย 1 กรณีที่สามารถหาคำตอบได้ทันที (base case หรือ simple case)
- ในขณะที่กรณีอื่นๆสามารถกำหนดได้ใหม่ (redefine) ในรูปที่ง่ายขึ้นกว่าเดิม เรียกว่า recursive case
- ปัญหาสามารถลดรูปลงมาสู่ simple case
 - *If this is a simple case*
solve it
 - else*
redefine the problem using recursion

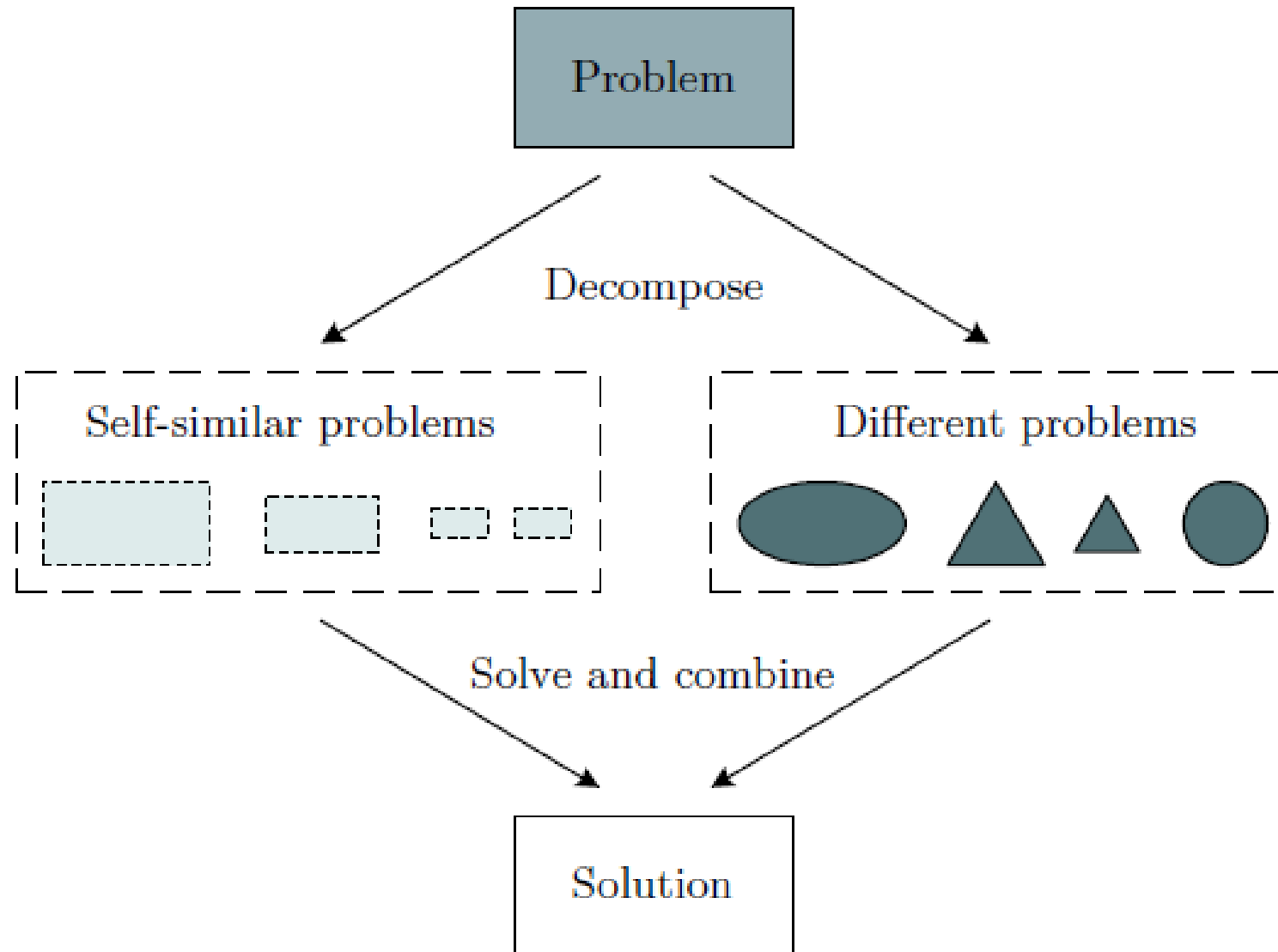
เงื่อนไขในการหยุดการเวียนบังเกิด (Terminating condition)

- recursive functions จะต้องมียุเงื่อนไขหยุดการทำงานอย่างน้อย 1 เงื่อนไขเสมอ
 - โดยปกติ จะเป็นเงื่อนไขของกรณีพื้นฐาน base case
- หากไม่มี terminating condition ฟังก์ชันอาจทำงานไม่หยุดเลย
 - เช่น ฟังก์ชันการคูณ if statement “`if (n == 1) ...`” คือ terminating condition.



การออกแบบฟังก์ชันเวียนบังเกิด (Designing recursive function)

- 1) หาขนาดของปัญหา (n)
- 2) กำหนด Base case
- 3) แบ่งปัญหาออกเป็นปัญหาย่อยที่มีโครงสร้างเหมือนปัญหาดั้งเดิม
- 4) กำหนด Recursive case ด้วยหลักการอุปนัย
- 5) ทดสอบโปรแกรม



ตัวอย่าง (Example)

```
#include <stdio.h>
int sum(int n);

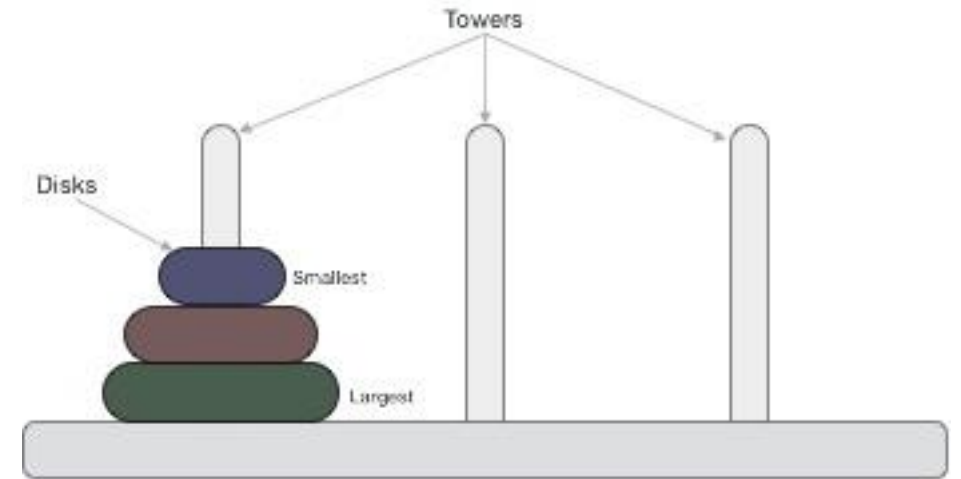
int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);
    result = sum(number);
    printf("sum = %d", result);
    return 0;
}
```

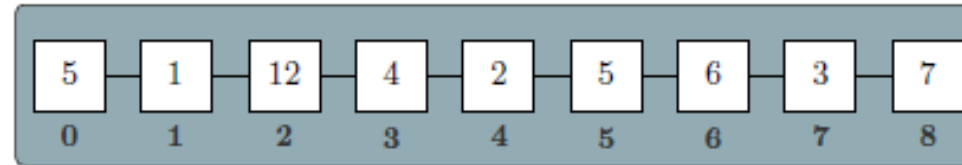
```
int sum(int num) {
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

รูปแบบของการเวียนเกิด (Types of recursion)

- Linear
 - Basic
 - Tail
- Multiple
 - Divine and Conquer
 - Puzzles and Fractal

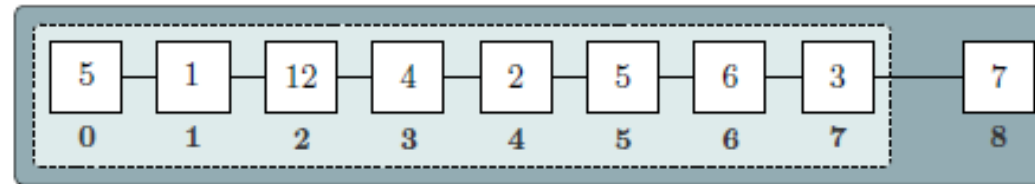


Original problem: $s(\mathbf{a}) = \mathbf{a}[0] + \mathbf{a}[1] + \dots + \mathbf{a}[n-1]$



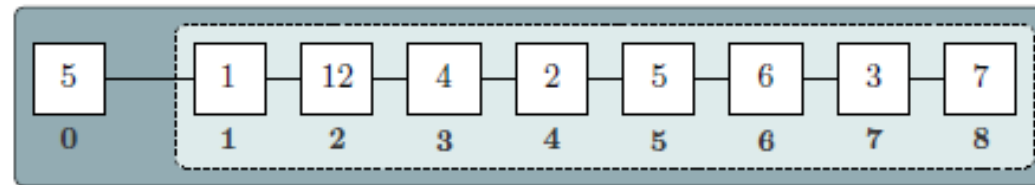
(a)

$s(\mathbf{a}) = s(\mathbf{a}[0:n-1]) + \mathbf{a}[n-1]$



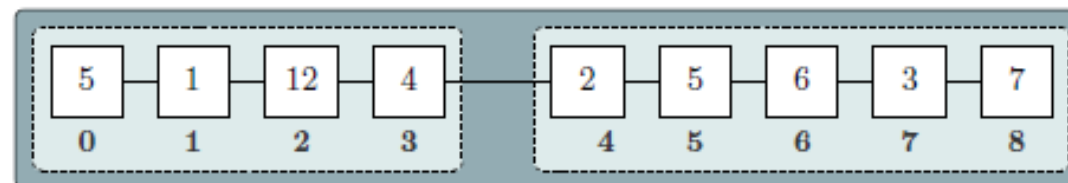
(b)

$s(\mathbf{a}) = \mathbf{a}[0] + s(\mathbf{a}[1:n])$



(c)

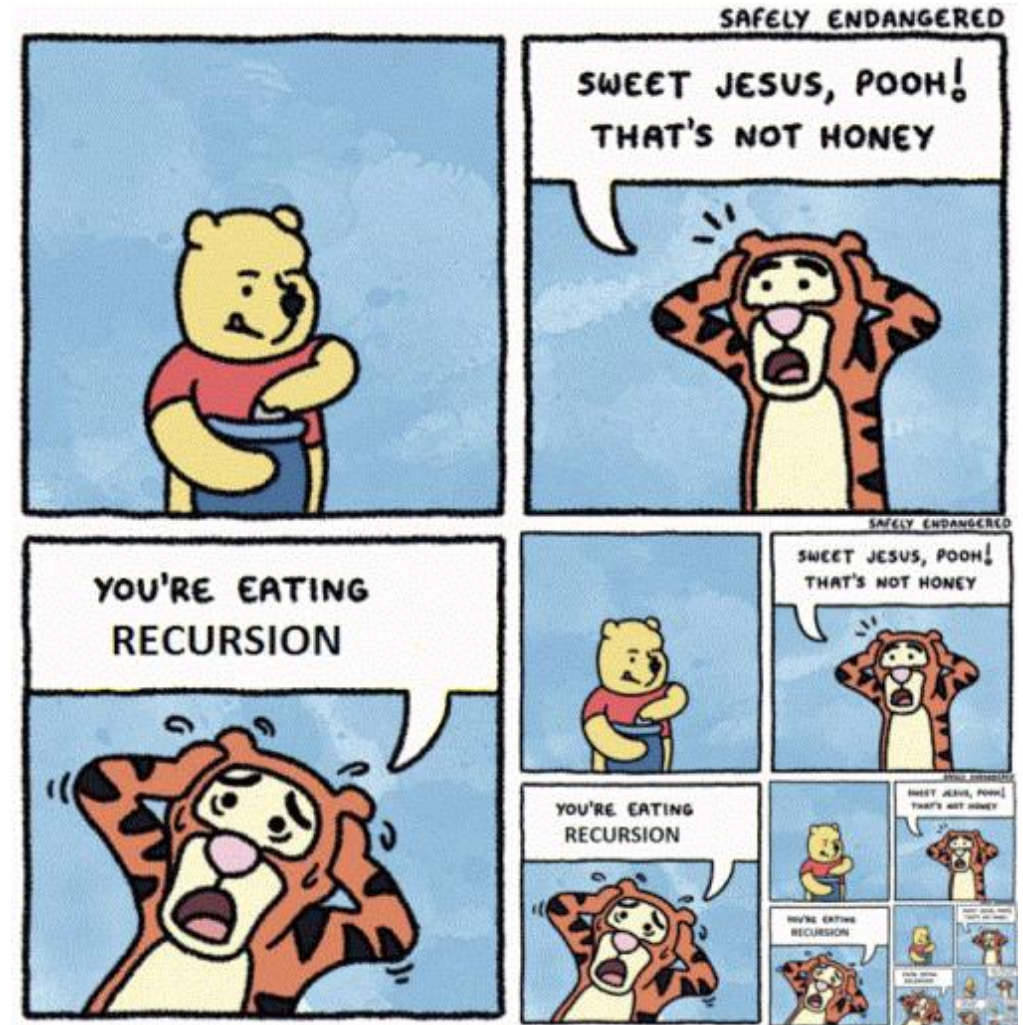
$s(\mathbf{a}) = s(\mathbf{a}[0:n/2]) + s(\mathbf{a}[n/2:n])$



(d)

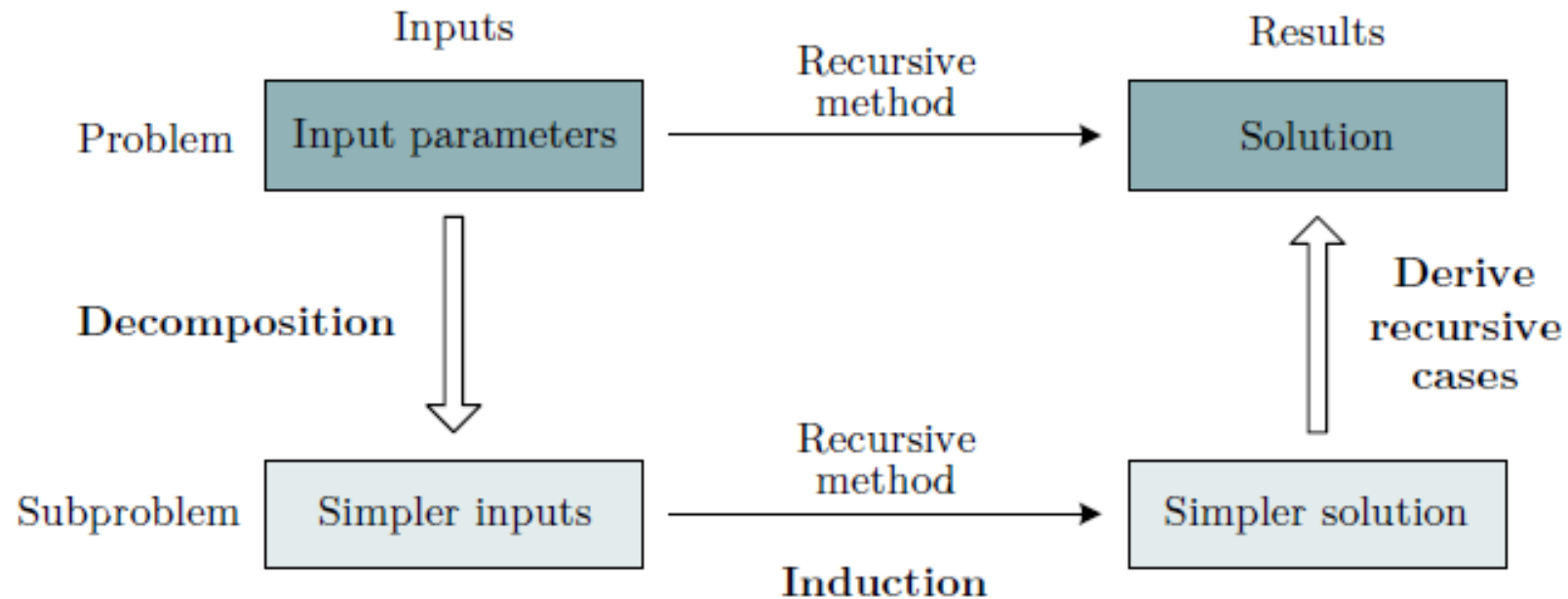
การเวียนเกิด VS. การวนซ้ำ (Recursive vs. Iterative)

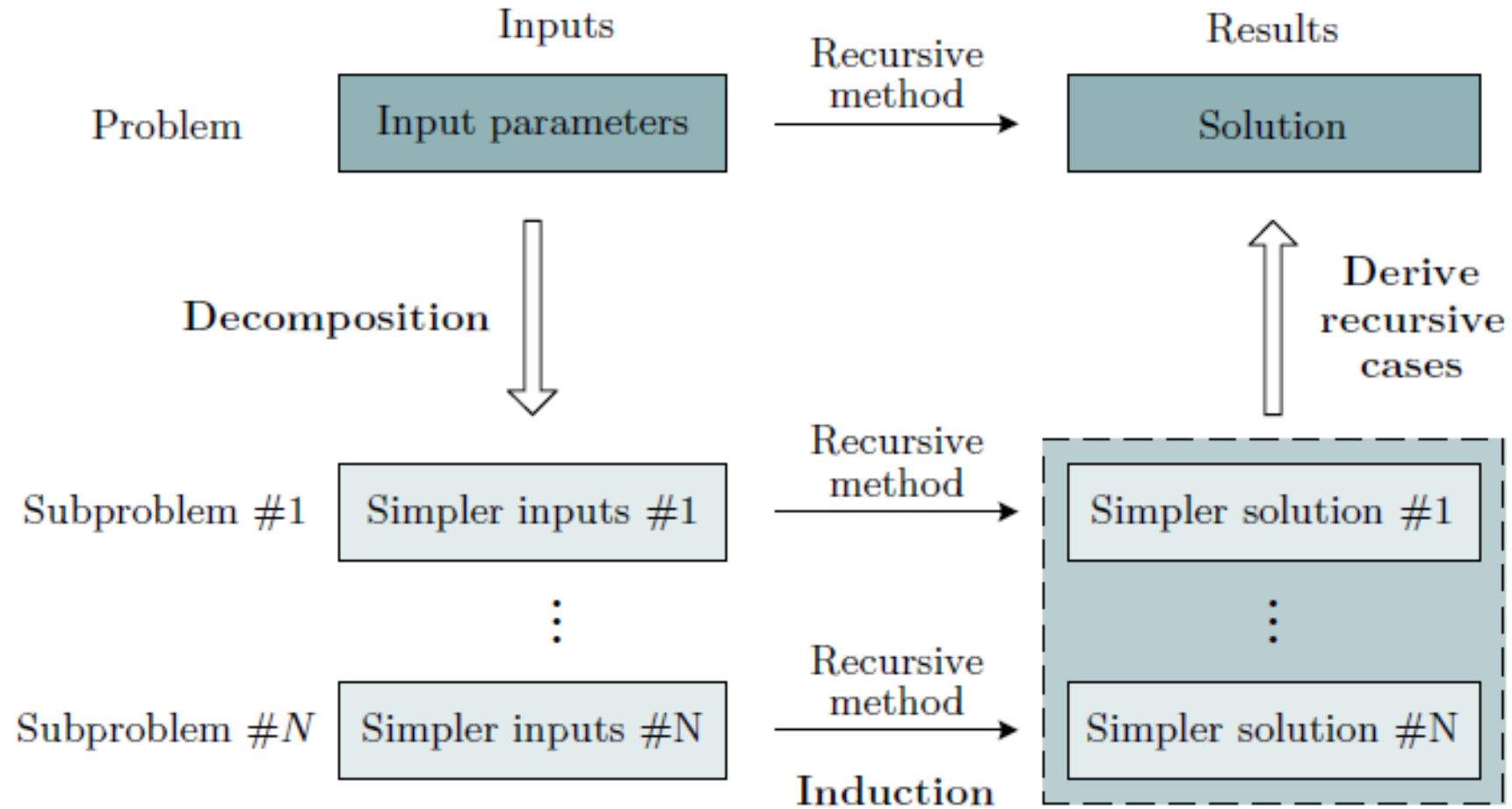
- การเวียนเกิด
 - เข้าใจง่ายกว่า
 - ออกแบบโปรแกรมง่ายกว่า
 - ใช้หน่วยความจำมากกว่า
 - ทำงานช้ากว่า



แผนภาพเวียนบังเกิด (Recursion diagram)

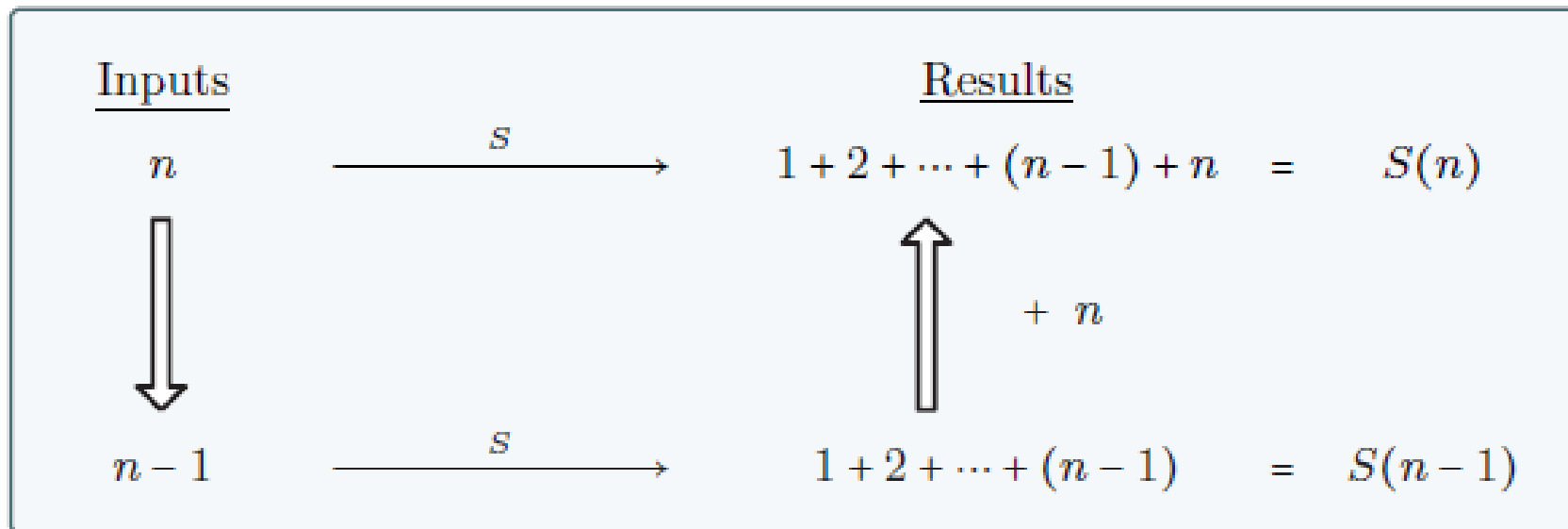
- ใช้ช่วยในการออกแบบฟังก์ชันเวียนบังเกิด





แบบฝึกหัด (Exercise)

- เขียนแผนภาพเวียนบังเกิดของการบวกเลขจำนวนเต็มตั้งแต่ 1 ถึง n



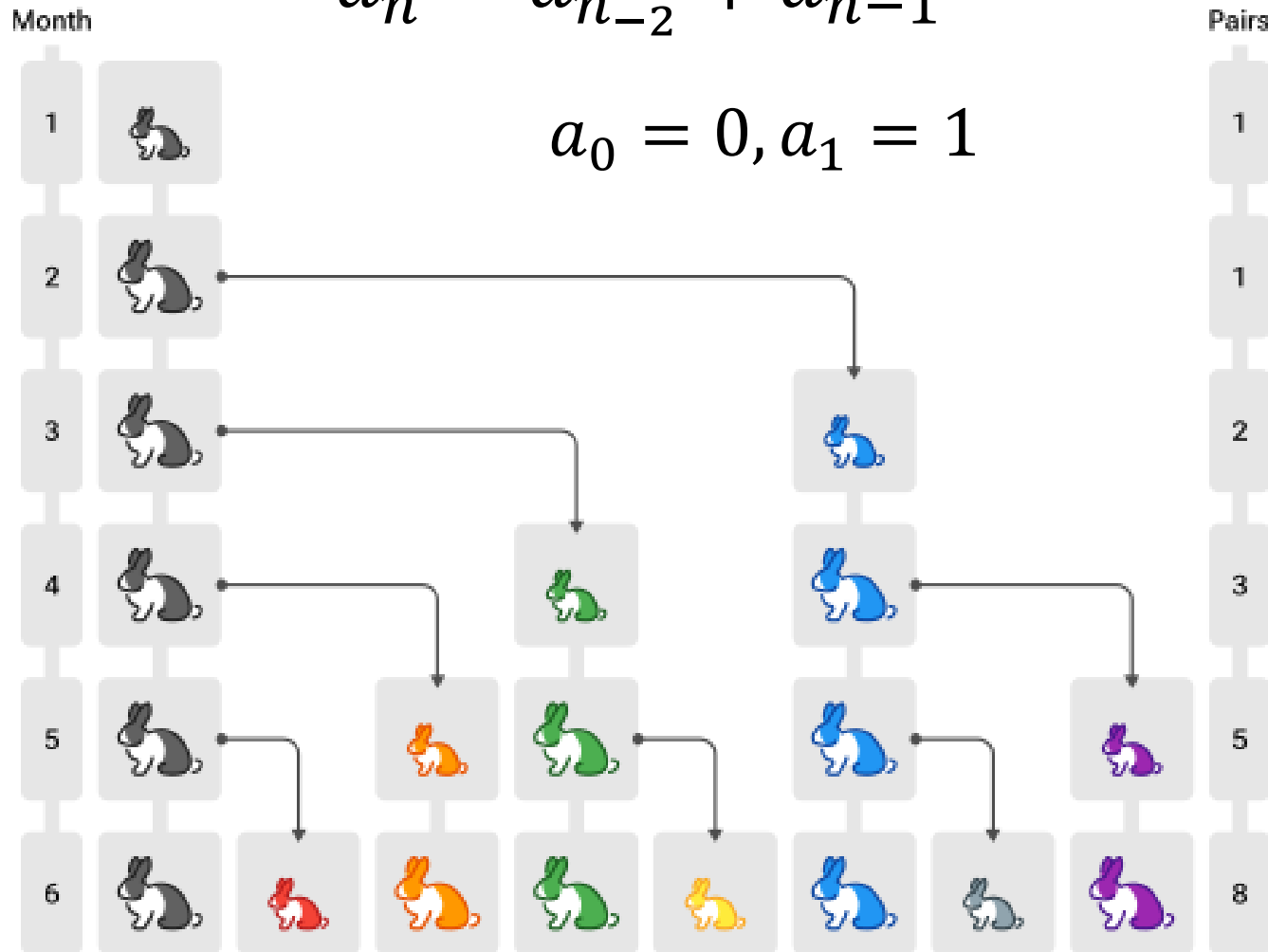
แบบฝึกหัด (Exercise)

- เขียนแผนภาพเวียนบังเกิดของการคำนวณดอกเบี้ยทบต้น เมื่อมีเงินต้น b บาท อัตราดอกเบี้ย $r\%$ ต่อปี และเวลาผ่านไป n ปี

Fibonacci

$$a_n = a_{n-2} + a_{n-1}$$

$$a_0 = 0, a_1 = 1$$



Golden Ratio

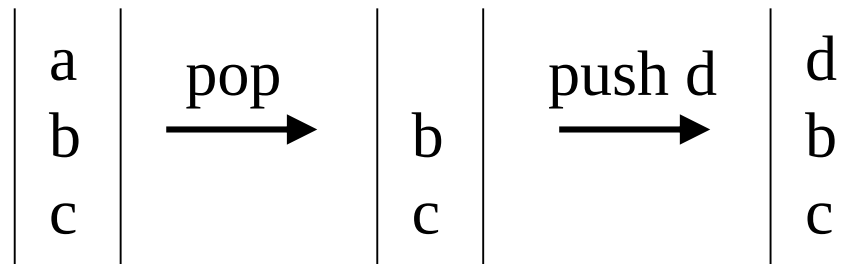


แบบฝึกหัด (Exercise)

- เขียนแผนภาพเวียนบังเกิดของการคำนวณพจน์ที่ n ของลำดับ Fibonacci

How C Maintains the Recursive Steps

- ภาษา C เก็บรักษาค่าตัวแปรโดยใช้ **stack**
 - stack โครงสร้างข้อมูลที่จัดการข้อมูลในรูปแบบ LIFO
 - 2 operations คือ push และ pop



How C Maintains the Recursive Steps

- ทุกครั้งที่มีการเรียกฟังก์ชัน สถานะปัจจุบันของผู้เรียก (caller function) ได้แก่ parameters, local variables, and memory address เป็นต้น จะถูก push ลงใน stack
- เมื่อฟังก์ชันที่ถูกเรียกประมวลผลเสร็จ ค่าสถานะต่าง ๆ ของ caller function จะถูกนำกลับมา โดยการ pop จาก stack.

How to Trace Recursive Functions

- การ trace และ debug ฟังก์ชัน recursive ไม่ง่าย !!!
 - เช่น ถ้ามีการเรียกตัวเองนับร้อยครั้ง การ set break point หรือการ trace แบบ step-by-step จะน่าเหนื่อยมาก
- วิธีการที่นิยมคือ การเพิ่มคำสั่ง print เข้าไปเพื่อดูผลของการทำงานในแต่ละ recursive steps.

```
1.  /*
2.   * *** Includes calls to printf to trace execution ***
3.   * Performs integer multiplication using + operator
4.   * Pre:  m and n are defined and n > 0
5.   * Post: returns m * n
6.   */
7.  int
8.  multiply(int m, int n)
9.  {
10.     int ans;
11.
12.     printf("Entering multiply with m = %d, n = %d\n", m, n);
13.
14.     if (n == 1)
15.         ans = m;      /* simple case */
16.     else
17.         ans = m + multiply(m, n - 1); /* recursive step */
```

Watch the input arguments passed into each recursive step.

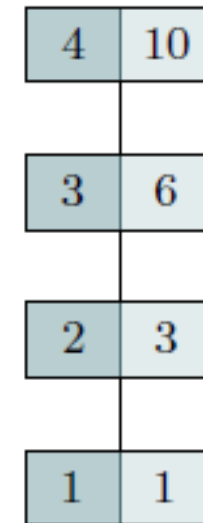
(continued)

ต้นไม้เวียนบังเกิด (Recursion tree)

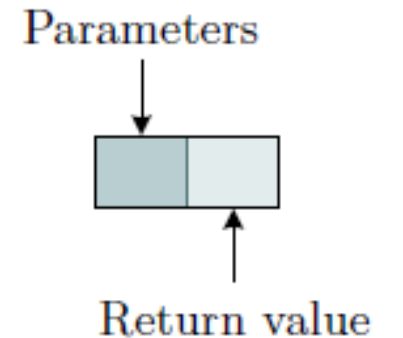
- ใช้ช่วยในการตรวจสอบ / ติดตามการทำงานของฟังก์ชันเวียนบังเกิด

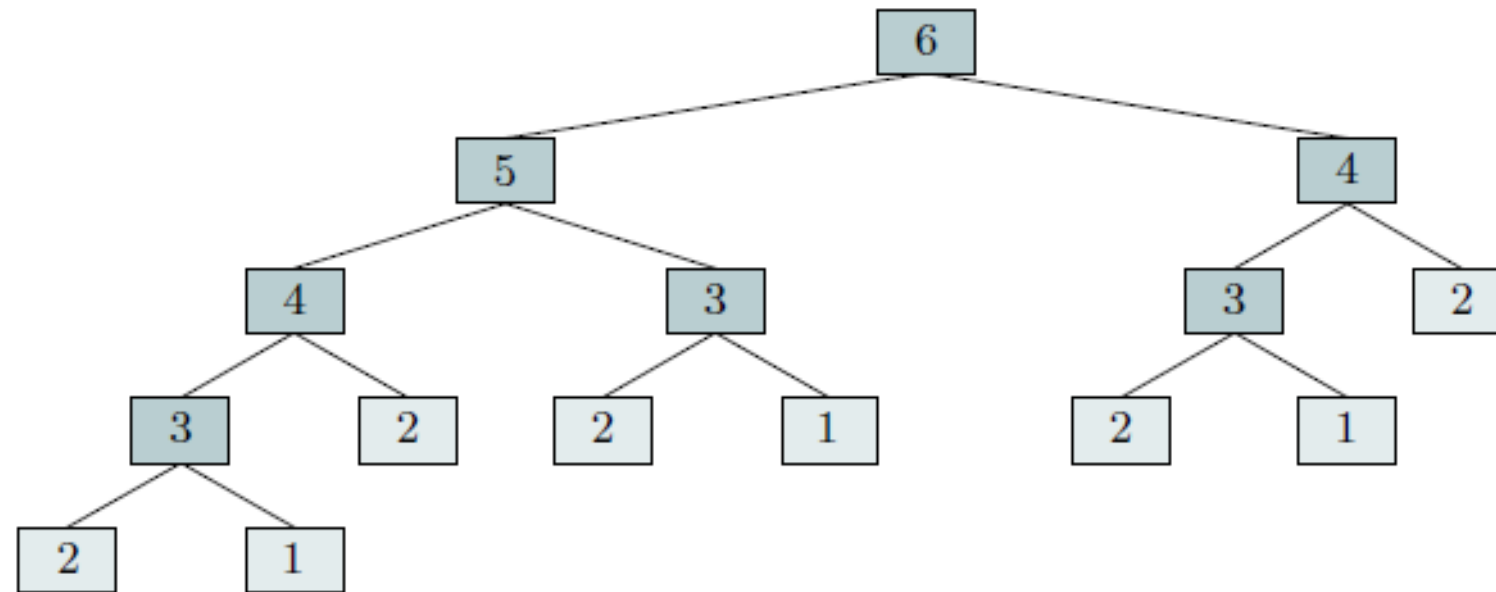


Recursion tree

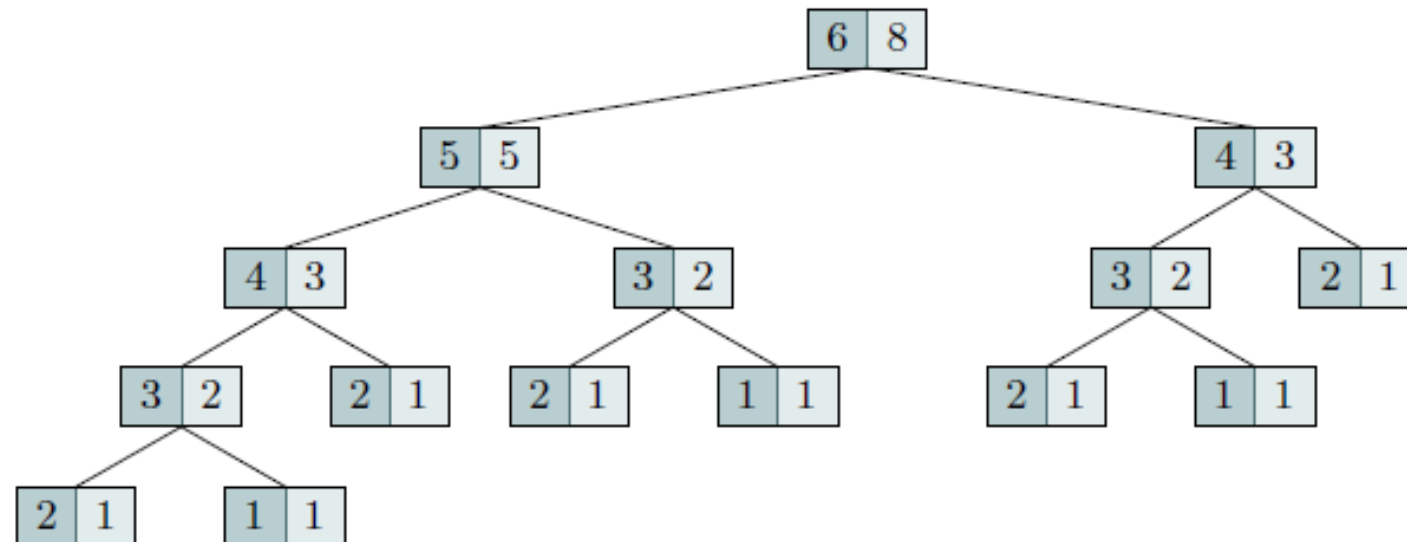


Activation tree





Fibonacci



Recursive Function:

นับจำนวนตัวอักษรที่ต้องการใน string

- We can count the number of occurrences of a given character in a string.
 - e.g., the number of 's' in "Mississippi" is 4.

```
1.  /*
2.   * Count the number of occurrences of character ch in string str
3.   */
4.  int
5.  count(char ch, const char *str)
6.  {
7.
8.      int ans;
9.
10.     if (str[0] == '\0')                /* simple case */
11.         ans = 0;
12.     else                                /* redefine problem using recursion */
13.         if (ch == str[0])               /* first character must be counted */
14.             ans = 1 + count(ch, &str[1]);
15.         else                             /* first character is not counted */
16.             ans = count(ch, &str[1]);
17.
18.     return (ans);
19. }
```

A Recursive Function: Reverses Input Words (1/2)

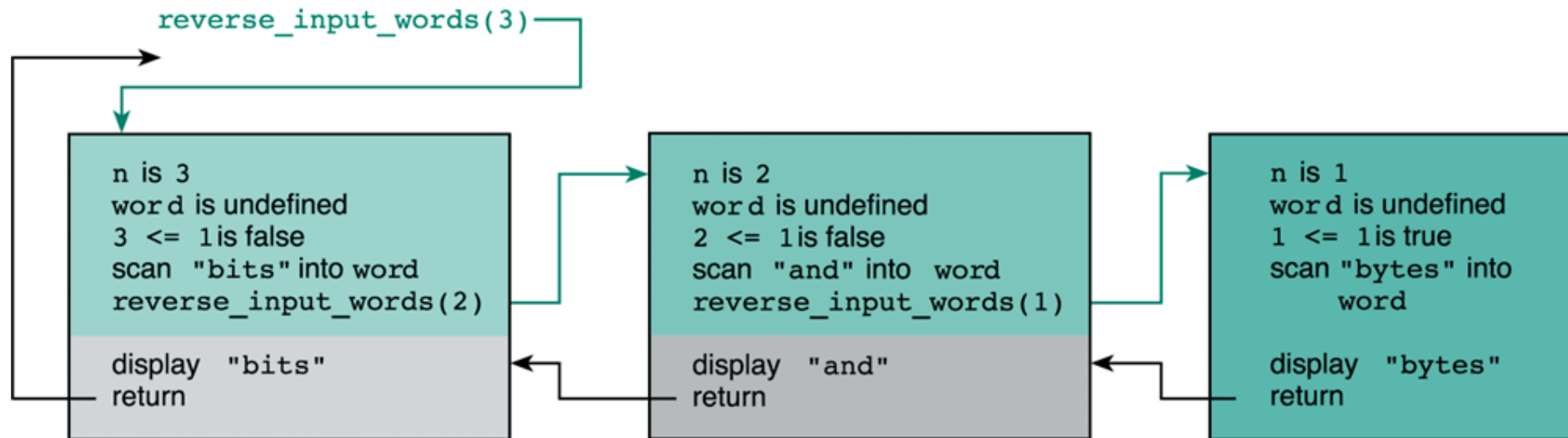
- The recursive concept can be used to reverse an input string.
 - It can also be done without recursion.

```
1.  /*
2.   *   Take n words as input and print them in reverse order on separate lines.
3.   *   Pre: n > 0
4.   */
5.  void
6.  reverse_input_words(int n)
7.  {
8.      char word[WORDSIZ]; /* local variable for storing one word */
9.
10.     if (n <= 1) { /* simple case: just one word to get and print */
11.
12.         scanf("%s", word);
13.         printf("%s\n", word);
14.
15.     } else { /* get this word; get the rest of the words in
16.              reverse order; then print this word */
17.
18.         scanf("%s", word);
19.         reverse_input_words(n - 1);
20.         printf("%s\n", word);
21.     }
22. }
```

The scanned word will not be printed until the recursion finishes.

The first scanned word is last printed.

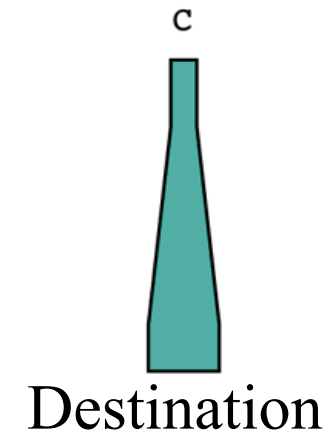
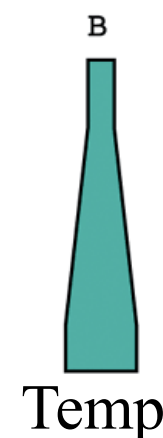
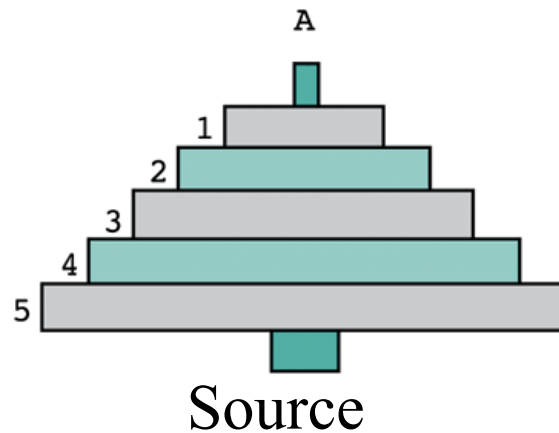
A Recursive Function: Reverses Input Words (2/2)



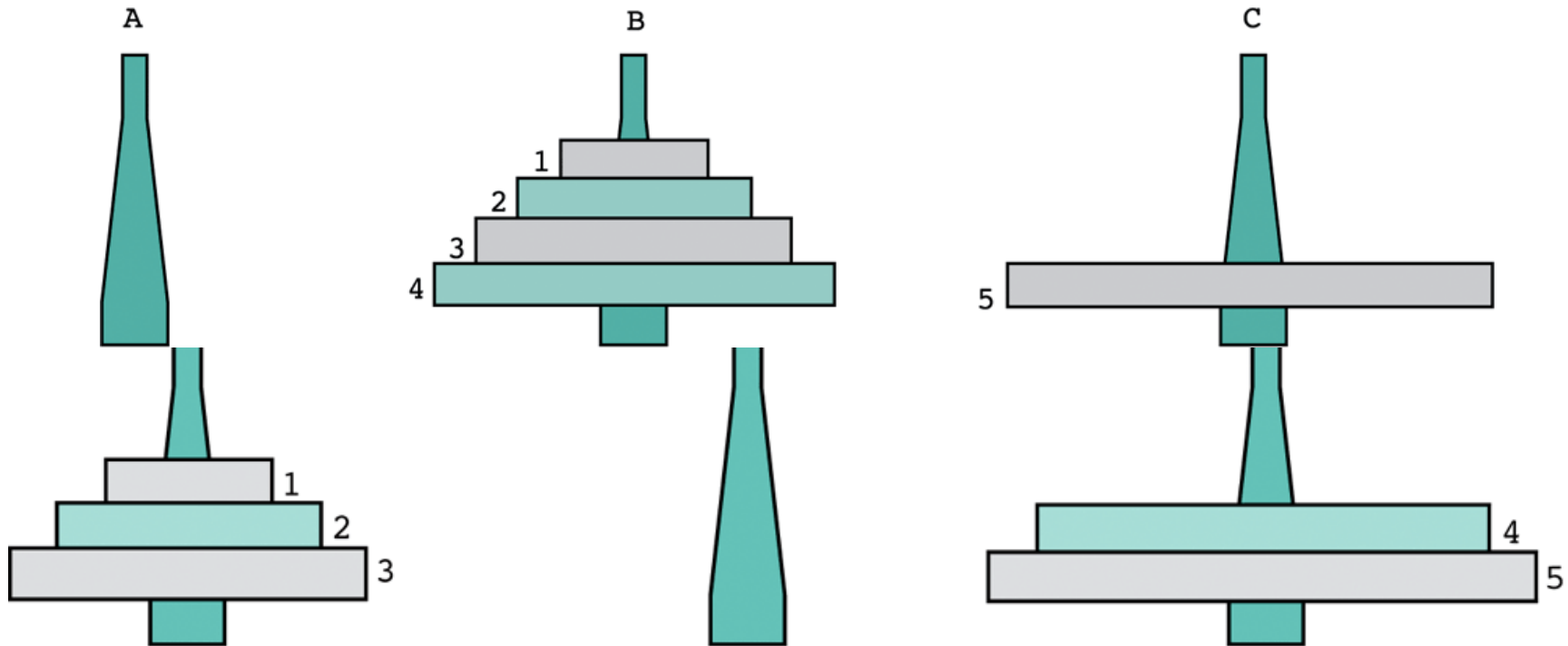
- Note that the recursive function is just an alternative solution to a problem.
 - You can always solve the problem without recursion.

หอคอยแห่งฮานอย (Tower of Hanoi) (1/3)

- เชื่อว่าเป็นการละเล่นอย่างหนึ่งของพระสงฆ์ในเมืองฮานอย ประเทศเวียดนาม
- จุดประสงค์ของการละเล่นนี้คือ ย้ายแผ่นจานทั้งหมด n จานจากเสาต้นทาง (source) ไปยังเสาปลายทาง (destination) โดยจะมีเสาทดให้ 1 เสา
- แผ่นจานแต่ละแผ่นจะมีขนาดต่างกัน
- กฎของการละเล่นนี้คือ
 - ย้ายแผ่นจานได้ทีละ 1 แผ่น
 - ห้ามให้แผ่นจานขนาดใหญ่กว่าทับแผ่นจานขนาดเล็กกว่า
 - สมมติให้มี 3 เสา



หอคอยแห่งฮานอย (Tower of Hanoi) (2/3)



หอคอยแห่งฮานอย (Tower of Hanoi) (3/3)

- ปัญหานี้สามารถแก้ได้ง่ายเมื่อใช้ recursive

- Algorithm:

if n is 1 then

 move disk 1 from the source tower to the destination tower

else

1. move $n-1$ disks from the source tower to the temp tower.
2. move disk n from the source tower to the destination tower.
3. move $n-1$ disks from the temp tower to the source tower.

Lab

- เขียนโปรแกรมเพื่อหาค่าของการบวกเลขจำนวนเต็มตั้งแต่ 1 ถึง n , $n \leq 1000$
- เขียนโปรแกรมเพื่อหาค่าของพจน์ที่ n ในลำดับ Fibonacci ด้วยวิธีเวียนบังเกิด เมื่อ $n \leq 40$

สรุปเรื่องการเวียนบังเกิด

- โปรแกรมเข้าใจง่าย
- โปรแกรมมีขนาดสั้น จำนวนบรรทัดน้อย
- สามารถแก้ปัญหาบางประเภทได้อย่างรวดเร็ว
- มีลักษณะคล้ายวงวน (Repetition)
- หยุดทำงานเมื่อเงื่อนไขเป็นจริง
- ใช้หน่วยความจำเยอะ
- ทำงานช้ากว่าวงวน

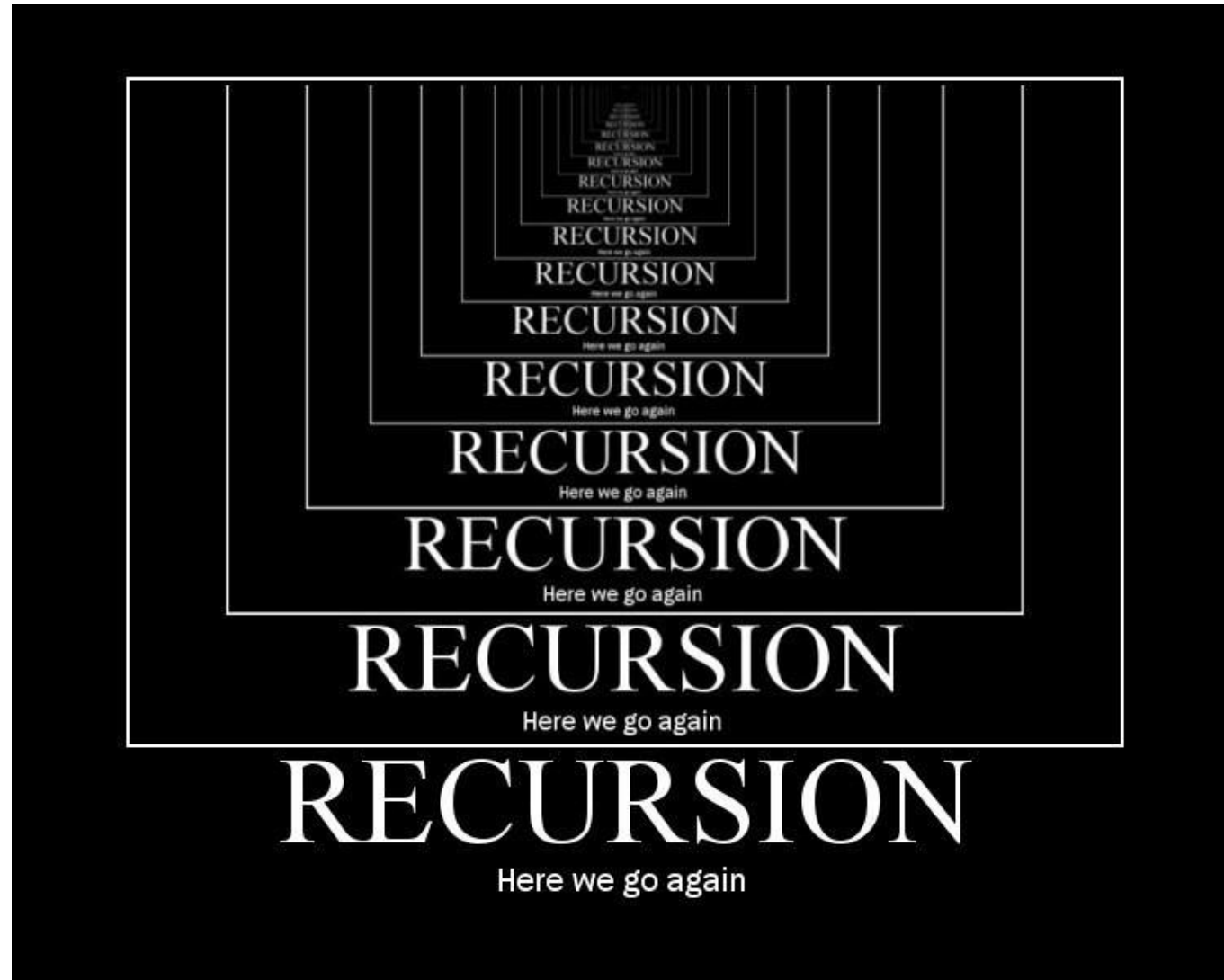




แบบฝึกหัดเขียนโปรแกรม (Programming exercises)

- เขียนโปรแกรมเพื่อหาค่า $n!$ ด้วยวิธีเวียนบังเกิด เมื่อ $n \leq 20$
 - เมื่อ $n! = n * (n-1)!$ และ $0! = 1$
- เขียนโปรแกรมเพื่อคำนวณดอกเบี้ยทบต้น เมื่อมีเงินต้น b บาท อัตราดอกเบี้ย $r\%$ ต่อปี และเวลาผ่านไป n ปี ($0 < b < 10^6$, $r \leq 10$, $n \leq 100$)

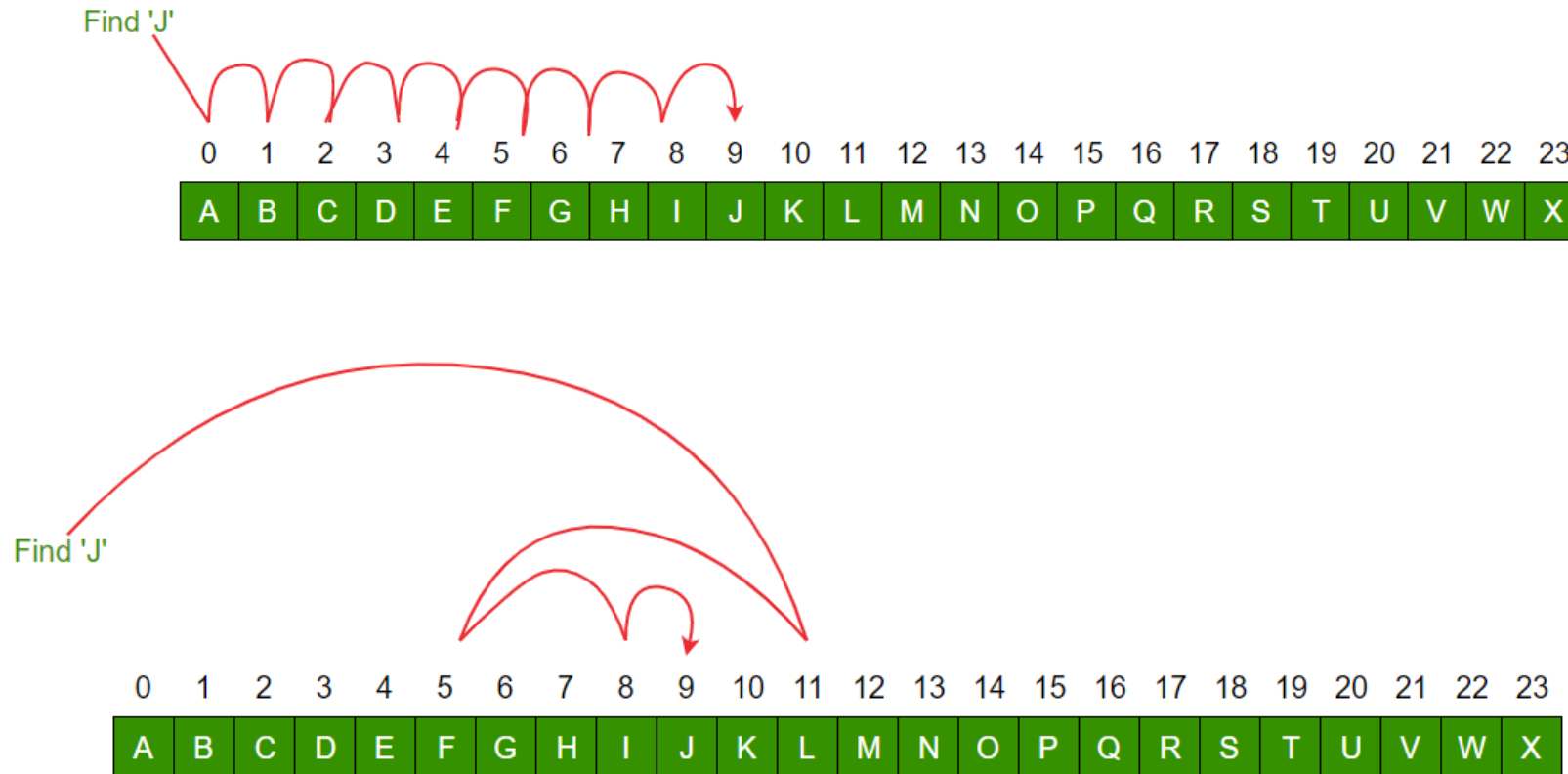
ทบทวน (Recitation)



โจทย์ปัญหาเพิ่มเติม



Binary search



- เขียนโปรแกรมเพื่อค้นหาจำนวนเต็ม x ในอาร์เรย์ของจำนวนเต็มขนาด n ตัวด้วยวิธีการ Binary search ด้วยวิธีเวียนบังเกิด $n \leq 1000$

Bubble Sort

Algorithm BubbleSort(A[0..n-1])

```
for i ← 0 to n-2 do
  for j ← 0 to n-2-i do
    if A[j+1] < A[j]
      swap A[j] and A[j+1]
```

Example:



Merge Sort

- การเรียงลำดับแบบผสาน
- อาศัยวิธี **Divide and conquer**
- โดยการแบ่งแถวลำดับออกเป็น 2 ชุดย่อย ชุดทางซ้ายและชุดทางขวาขนาดพอๆ กัน
- นำข้อมูลทั้ง 2 ชุดมาเรียงลำดับ
- นำผลของสิ่งที่ได้มาผสานกัน ผลลัพธ์สุดท้ายจะเป็นผลลัพธ์ที่เรียงลำดับ

Merge Sort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Merge Sort

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)
//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
 $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
 if $B[i] \leq C[j]$
 $A[k] \leftarrow B[i]; i \leftarrow i + 1$
 else $A[k] \leftarrow C[j]; j \leftarrow j + 1$
 $k \leftarrow k + 1$
if $i = p$
 copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Merge Sort : Divide & Conquer

